

INFOMCV Assignment 4

Majo Bednár (2439905), Angel Bu Tong Mei (9682120) (Group 98)

Description of your implementation

Model	[1, 343]	--
└─Conv2d: 1-1	[1, 16, 112, 112]	448
└─BatchNorm2d: 1-2	[1, 16, 112, 112]	32
└─MaxPool2d: 1-3	[1, 16, 56, 56]	--
└─Conv2d: 1-4	[1, 32, 56, 56]	4,640
└─BatchNorm2d: 1-5	[1, 32, 56, 56]	64
└─MaxPool2d: 1-6	[1, 32, 28, 28]	--
└─Conv2d: 1-7	[1, 64, 28, 28]	18,496
└─BatchNorm2d: 1-8	[1, 64, 28, 28]	128
└─MaxPool2d: 1-9	[1, 64, 14, 14]	--
└─Conv2d: 1-10	[1, 64, 14, 14]	36,928
└─BatchNorm2d: 1-11	[1, 64, 14, 14]	128
└─MaxPool2d: 1-12	[1, 64, 7, 7]	--
└─Conv2d: 1-13	[1, 32, 7, 7]	18,464
└─BatchNorm2d: 1-14	[1, 32, 7, 7]	64
└─Flatten: 1-15	[1, 1568]	--
└─Linear: 1-16	[1, 512]	803,328
└─Dropout: 1-17	[1, 512]	--
└─Linear: 1-18	[1, 343]	175,959
Total params: 1,058,679		
Trainable params: 1,058,679		
Non-trainable params: 0		
Total mult-adds (Units.MEGABYTES): 43.79		
Input size (MB): 0.15		
Forward/backward pass size (MB): 5.85		
Params size (MB): 4.23		
Estimated Total Size (MB): 10.24		

Apart for the sigmoid for the output, we used ReLU activation functions. ReLU is computationally efficient. Moreover, given our network is (relatively) deep, ReLU does not shrink gradient compared to sigmoid or tanh, thus being more effective against vanishing gradient problem.

We used max pooling, as it focuses more on the discriminative features compared to average pooling. In theory, the network can capitalize on this in detecting objects.

We used batch normalization after every convolution layer for regularization and faster learning. To regularize the first dense layer, we applied dropout with $p=0.5$. High regularization is desirable due to the small number of training samples which the network can otherwise easily overfit to.

Description of data preparation

We resize the images to 112x112 with pytorch's resize transformation function and convert them to tensors.

We adapt the bounding boxes as follows. First, we determine the center of each bounding box and normalize it to range $[0, 1]$ relative to the image (by dividing by either a width or height of the image). Then, we determine to which cell in a 7x7 grid the center belongs to, by multiplying with the grid size (7) and rounding down. Finally, we normalize the center with respect to the grid cell, by

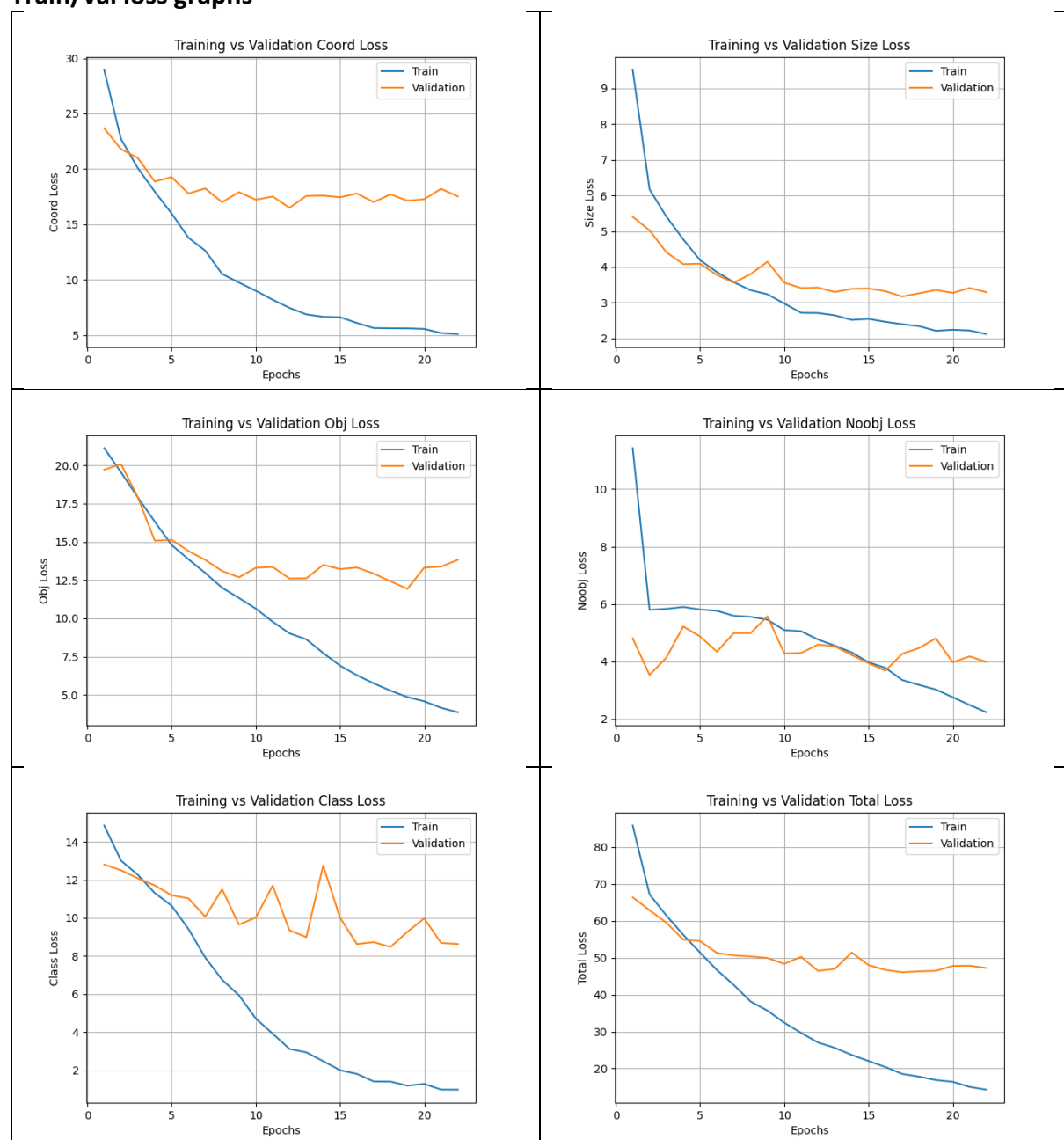
multiplying with the grid size and subtracting the cells ordering value. The width and height (after they are determined from the original bounding box corners) are simply normalized with respect to the image to range [0, 1] by dividing by the respective dimension.

The objectness/confidence value is set to 1 for a cell if a center of the bounding box falls into the cell, and the corresponding class label is set to 1 as well. When there is no object whose center falls into a grid cell, all values (bounding box, objectness, class labels) are initialized to zeros.

Description of your training hyperparameter

Hyperparameter	Value
Learning rate	0.001
Optimizer	Adam
Batch size	32

Train/val loss graphs



Training, validation and test mAP

Setting	mAP
Training	0.621
Validation	0.684
Test	0.637

Confusion matrix

		Actual		
		Cat	Dog	None
Predicted	Cat	163	8	29
	Dog	10	354	60
	None	65	138	X

Link to your model weights

[Model weights](#)

Discuss your results

All loss components decrease steadily for the training set and are quite low by twenty-second epoch, when the model stops training. For validation set, the loss components decrease as well, but not the same extent and are more jittery. The total validation loss converges at around epoch 17, when it is the lowest (early stopping patience was set to 5 epochs). The discrepancy between the loss components of training and validation sets suggests a slight overfitting to the training set.

Interestingly, however, mAP was the highest for validation set (0.684), then for test set (0.637) and lowest for test set (0.621). Although this is caused partly by the variable nature of images across the sets, it indicates that the model generalizes reasonably well, and differences between training and validation losses were sufficiently small.

As you can see in the confusion matrix, the model is good at classification (only 10/173 cats were misclassified as dogs and only 8/354 dogs were misclassified as cats). However, the weaknesses of the model lie mainly in missing object detections and (to a slightly lower extent) detecting objects when there are none. Specifically, the model missed 65 cats and falsely detected 29 cats. Similarly, the model missed 138 dogs and falsely detected 60 dogs.

Choice tasks

Choice 6: Data Augmentation (Photometric Techniques)

To improve model generalization without invalidating our 7x7 bounding box grid targets, we implemented a photometric-only data augmentation pipeline using torchvision.transforms. Applied exclusively to the training set, we utilized three spatial-preserving techniques:

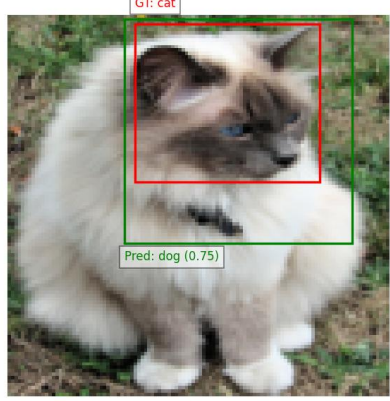
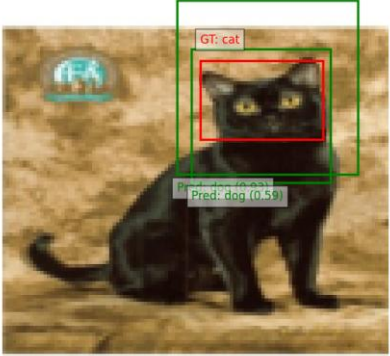
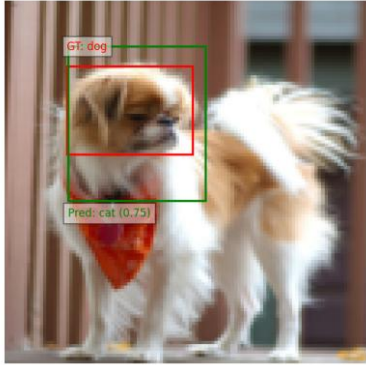
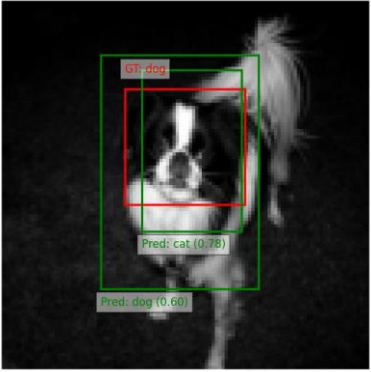
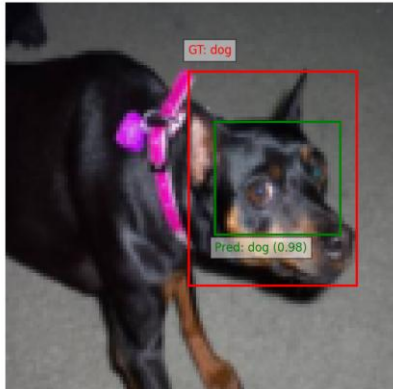
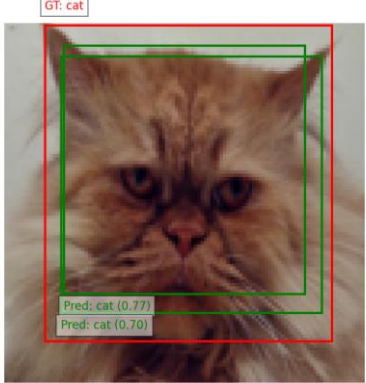
1. **Color Jitter:** Random adjustments to brightness, contrast, saturation, and hue.
2. **Gaussian Blur:** Simulating low-resolution or out-of-focus camera conditions.
3. **Random Autocontrast:** Maximizing dynamic range by remapping pixel extremes.

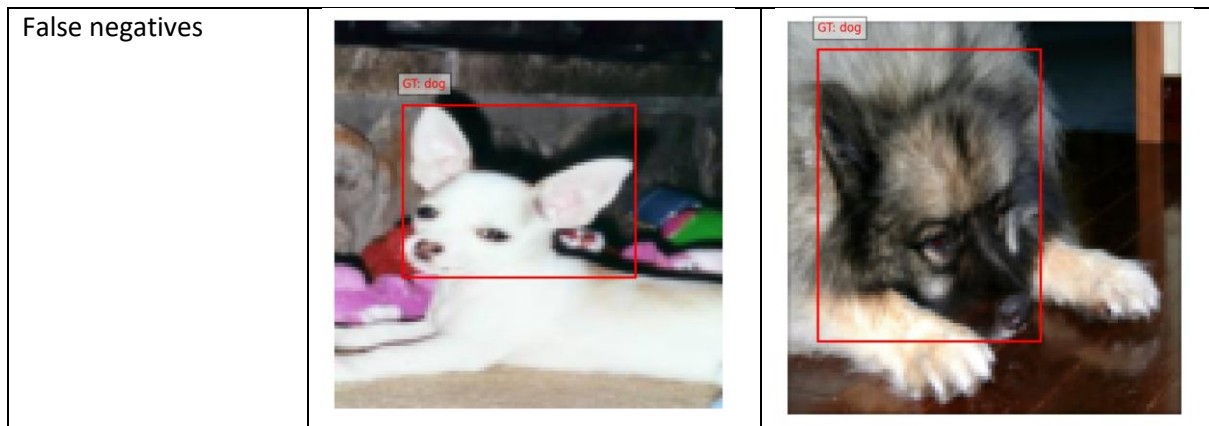
Model	mAP @ 0.5 Test set
Baseline	0.637
Augmented	0.587

Analysis:

Restricting our pipeline to photometric transformations forced the model to learn illumination- and clarity-invariant features while keeping the spatial bounding box geometry perfectly intact. As shown in the table above, this approach did not help much. Although the training is stochastic and the result may be worse purely by chance, it is possible that the model was learning too much of the features of the augmented images and failed to generalize to the same extent to the non-augmented test images.

Choice 7: Misclassifications and Misdetections:

Cat to dog misclassifications		
Dog to cat misclassifications		
False positives		



From the examples of misclassifications, we can see that there is some inter-class similarity. For example, the furry cat has similar features to the furry dog. These exemplars are clearly where the boundary between the two classes meets.

The main reason for getting false positives arise from the inaccurate predictions of bounding boxes, thus not passing the intersection over union threshold. This error also stems from predicting multiple objects when there is only one present.

The reason for getting false negatives is also from inaccurate predictions of bounding boxes (the dog in the false positives row is also a false negative, as it was not matched by any bounding box prediction), or (as seen by the examples), when the model simply does not detect the object at all.

Choice 8: Non-Maximum Suppression

Because our mini-YOLO architecture predicts a bounding box for every grid cell, it sometimes outputs multiple redundant boxes for a single localized object. To resolve this, we implemented Non-Maximum Suppression (NMS) using `torchvision.ops.nms`. We applied a greedy, per-class approach with an IoU threshold of 0.45. Crucially, this was integrated strictly as a post-processing step during inference. By isolating NMS from the training loop, we ensured the model's loss calculations and spatial learning remained entirely unaffected.

mAP no NMS: 0.656 | mAP with NMS: 0.654

Confusion matrix no NMS

		Actual		
		Cat	Dog	None
Predicted	Cat	166	14	40
	Dog	5	357	61
	None	67	129	X

Confusion matrix with NMS

		Actual		
		Cat	Dog	None
Predicted	Cat	167	13	28
	Dog	4	354	43
	None	67	133	X

Any GenAI prompt used

(Mention tool and prompt, and how the output was used. No limit on space, add all prompts you performed.)

(ChatGPT) `TypeError: conv2d() received an invalid combination of arguments - got (tuple, Parameter, Parameter, tuple, tuple, tuple, int), but expected one of: *` (Tensor input, Tensor weight, Tensor bias = None, tuple of ints stride = 1, tuple of ints padding = 0, tuple of ints dilation = 1, int groups = 1) didn't match because some of the arguments have invalid types: (!tuple of (Tensor, Tensor), !Parameter!, !Parameter!, !tuple of (int, int)!, !tuple of (int, int)!, !tuple of (int, int)!, !int!) * (Tensor input, Tensor weight, Tensor bias = None, tuple of ints stride = 1, str padding = "valid", tuple of ints dilation = 1, int groups = 1) didn't match because some of the arguments have invalid types: (!tuple of (Tensor, Tensor), !Parameter!, !Parameter!, !tuple of (int, int)!, !tuple of (int, int)!, !tuple of (int, int)!, !int!) - Error while using data from the dataloader, we modified the collate function accordingly

(ChatGPT) How can I reshape a flat tensor to a one with specified dimensions? (Used the view method for reshaping raw output from the model)

(ChatGPT) If I have multidimensional tensor in pytorch, how can I filter out one element of the last dimension, for all other dimensions? (For computing losses)

(ChatGPT) There were some bugs with graphs – for fixing the bugs